Cache-Aside (Lazy Loading)

the app checks the cache, and fills it on a miss

Watch the 5-min video

> **In one line:** in cache-aside the application reads the cache first, and on a miss reads the database and stores the result with a TTL; on a write it updates the database and then deletes the key, because deleting is safer than updating the cache, while TTLs bound the small stale window that remains.

1 Who fills the cache?

You've added Redis next to your database. Who puts data into the cache, when, and what happens when the data changes? The most common answer, used in most systems you'll ever build, is **cache-aside**, also called **lazy loading**. The application talks to both the cache and the database; the cache is just storage on the side.

2 The read path

1. Ask the cache for the key.
2. **Hit**: return the cached value.
3. **Miss**: read from the database, store the result in the cache with a TTL, and return it.

Only data that's actually requested gets cached, which is why it's called *lazy*. The cache fills itself with the working set.

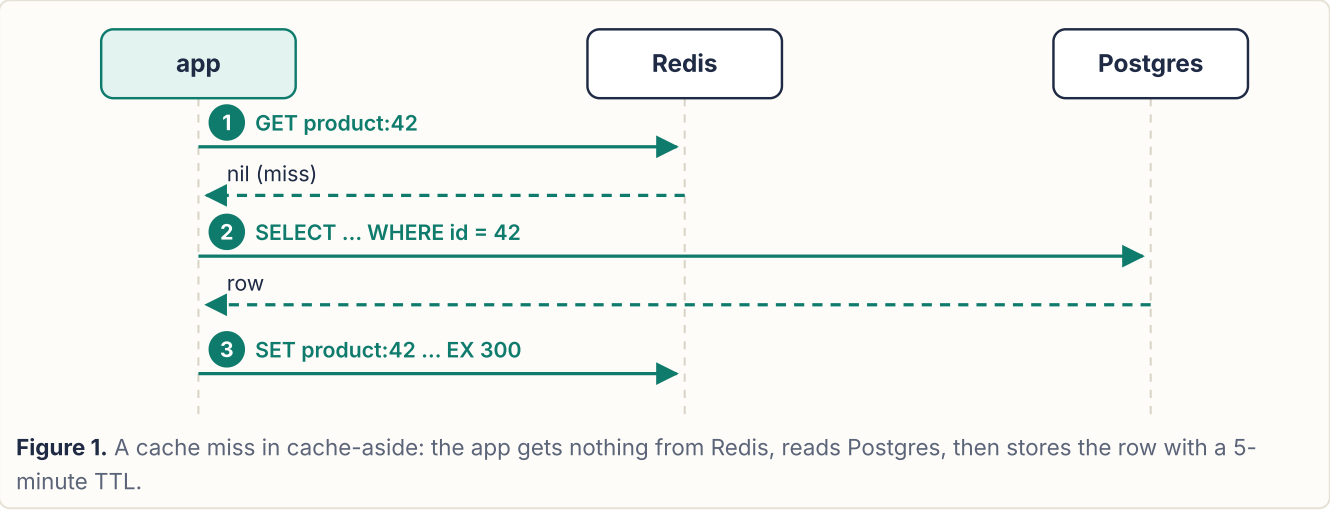


Figure 1. A cache miss in cache-aside: the app gets nothing from Redis, reads Postgres, then stores the row with a 5-minute TTL.

3 The write path: update the database, delete the key

On a write, the app updates the **database first**, then **deletes** the cache key. The next read misses, loads the fresh value, and repopulates the cache.

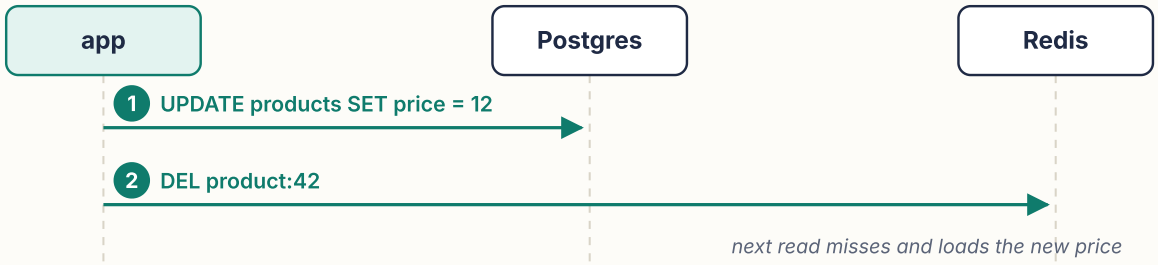


Figure 2. Writes go to the database, then the cache entry is deleted rather than updated.

WORKED EXAMPLE: A FASTAPI ENDPOINT

`GET /products/{id}` calls `redis.get(f"product:{id}")`. If there's a value, it parses the JSON and returns it. If not, it queries Postgres, calls `redis.set(key, json, ex=300)` (5 minutes) and returns the product. `PUT /products/{id}` writes to Postgres, then calls `redis.delete(key)`.

Ten lines of code, any cache product, and the database stays the source of truth.

4 Why delete instead of update?

Two writers can race. Writer A updates the database to price 10, writer B to price 12, but their cache updates arrive in the opposite order: the cache keeps 10, while the database says 12, until the TTL expires. Deleting avoids that: whatever the order, the next read loads the current value. Updating the cache also wastes work if nobody reads the key before it changes again.

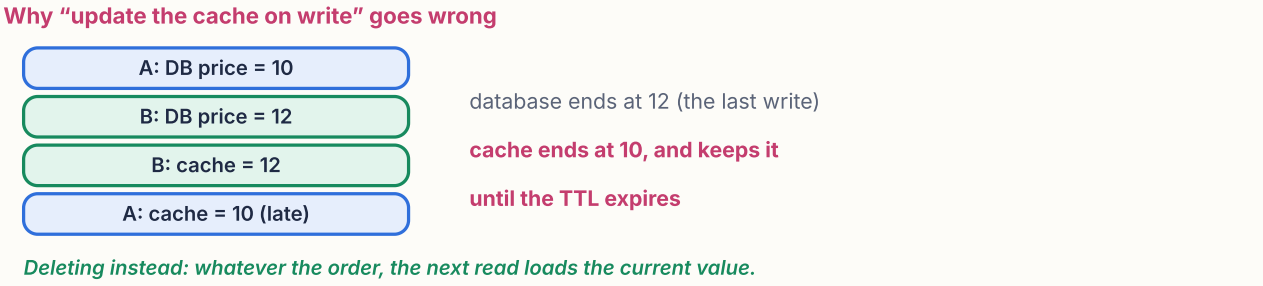


Figure 3. Two concurrent writers that both update the cache can leave it holding the older value.

5 The stale window that remains

Between the database write and the cache delete, readers see the old value: usually a few milliseconds. A rarer race remains too: a slow reader loads an *old* value from the database, the writer commits and deletes the key, and then the slow reader writes its old value into the cache. **TTLs bound how long any stale entry can live.** Some teams also **delete twice**: once immediately, and once again after a short delay (say 500 ms) to catch that race. Lesson 23 covers stronger options.

6 Failure modes

- **Cache down**: every request goes to the database. The database must survive it, or you need rate limiting and a circuit breaker around the cache.
- **Cold cache** after a restart or deploy: a burst of misses. Warm the hottest keys first.
- **Stampede**: many requests miss the same hot key at once and all query the database. A lock or request coalescing fixes it (lesson 24).

COMMON MISTAKES

- Deleting the cache key *before* the database write: a reader can refill the old value in between.
- No TTL, so a missed invalidation leaves wrong data forever.
- Caching "not found" without a short TTL, or not caching it at all and letting repeated misses hammer the database (lesson 25).

7 Key terms

Cache-aside: the app reads the cache and fills it on a miss

Source of truth: the database; the cache is a copy

Stale window: time the cache can show an old value

Lazy loading: cache only what's requested

Invalidation: deleting a cache entry after a change

Delayed double delete: delete again shortly after the write

8 Interview questions

Q1. Walk through cache-aside reads and writes. BASIC

On read: check the cache and return on a hit; on a miss, read the database, store the result with a TTL, and return it. On write: update the database, then delete the cache key.

Q2. Why delete rather than update on write? BASIC

Concurrent updates can leave the cache with an older value, while a delete always forces the next read to load the current data, and avoids caching data nobody reads.

Q3. What are the advantages of cache-aside? BASIC

Simple, works with any cache, caches only what's used, and the database stays the source of truth, so a cache failure only slows things down.

Q4. Redis goes down during peak. What happens with cache-aside, and how do you prepare? SCENARIO

Every read falls through to the database, which may be overloaded. Size the database for some miss traffic, put timeouts and a circuit breaker on cache calls, rate-limit or shed load, and warm the cache when it returns.

Q5. Describe a race that leaves stale data even with delete-on-write. TRICKY

A reader misses, reads the old row, pauses; a writer updates the row and deletes the key; the reader then writes the old row into the cache. TTLs bound it; a delayed second delete or versioned values reduce it.

Q6. Should you delete the key before or after the database write? TRICKY

After. Deleting first opens a gap where a reader misses, reads the old value from the database, and caches it before the write commits.

Next: SD-F-003 · Read-Through and Write-Through backendroadmap.com